k-Nearest Neighbors

DSC 340 Applied Machine Learning

Week 5, Day 2, March 7

The Plan

- Discuss kNN
- Demo in Python

Parametric methods

- Describes data with a few **parameters** (statistics concept)
  - Example: Normal distribution $N(\mu, \sigma^2)$: **mean** and **variance**
  - Small number of these relative to training set
- The **weight vector** and **bias** $(\mathbf{w}, w_0)$ in linear models ($m + 1$ params.)
  - linear regression
  - logistic regression
- Machine learning task: **learn the parameters**
- Then use to make **predictions**: method no longer needs training data

Nonparametric methods

- In contrast, **nonparametric methods** don't learn **parameters**.

*Note: They do have hyperparameters, as we will see.

- They **consult the training data itself** to make predictions.
- Motivation: **Histogram** (instead of normal distribution).
  - Similar, more advanced idea: **kernel density estimation.**
- Why? In many cases, datasets can't be suitably modeled …

  with any well-understood statistical distribution.
- **Consult training data directly to make predictions, guide decisions.**

Big Idea

- Analogy: Suppose you're a real estate agent.
- Prospective client asks you about putting their house on the market.
- How do you determine a reasonable sale price?
- **Search for comparables.** ("I'll get you some comps?")
- What that means: find **similar** houses **nearby** that sold **recently**.
  - "What are houses selling for in this neighborhood?"
  - Note that price prediction is a regression problem.
- Takeaway: **Estimate $\hat{y}$ by searching training data for similar examples.**

Nearest neighbor methods

- **Regression:** Find k similar examples: use **mean** to predict **value**
- **Classification:** Find k similar examples: use **mode** to predict **class**
  - The **mode** is **a most common** item in a set
  - Think of it as plurality **voting**: break ties randomly or however
- **Unsupervised** analogues: clustering method known as k-means
  - Method finds k clusters in the data: groups similar items together
- Note that k here could be any positive integer: 1, 3, 5, etc.: **1-NN**
- Question: Realtor uses experience and intuition to find similar houses
- **How do we define "similar items" so machine can search for them?**

Distance

- Training data in supervised learning: (x, y) examples
- For example, suppose you have the following dataset:

Height	Weight	1RM Squat	Position
6'8"	**340**	**750**	**Offensive lineman**
6'1"	**185**	**350**	**Defensive back**
5'10"	**225**	**420**	**Running back**

- Now you want to classify data with these features: **(6'10", 300, 680)**?
- Which one is most similar? How do you know?

Distance (more abstract)

- What about this dataset?

x_1	x_2	x_3	y
2	0	2	0
1	2	12	1
2	3	4	0

- Which of these is most similar to unclassified point (1, 2, 3)?
- Justify your reasoning.

Common distance metrics

- **Euclidean (L2):** Square root of sum of squared differences

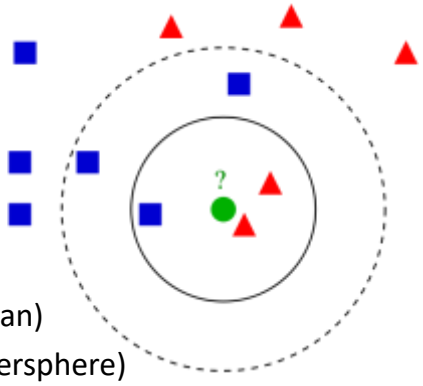
$$\|\mathbf{x} - \mathbf{x}'\|_2 = \sqrt{\sum_{j=1}^{m} \left(x_j' - x_j\right)^2}$$

- **Manhattan (L1):** Sum of absolute differences ("city block", "taxicab")

$$\|\mathbf{x} - \mathbf{x}'\|_1 = \sum_{j=1}^{m} \left|x_j' - x_j\right|$$

- **Chebyshev (max):** Greatest absolute difference along any coordinate
- **Hamming distance (L0):** Number of features that differ (DNA seq.)
- **Haversine distance:** Shortest path distance on a sphere (geolocation)

Mathematical foundation



- **Training data** $(\mathbf{X}, \mathbf{y})$
- Given a **new point** $\mathbf{x}$, **estimate** $\hat{y} \approx f(\mathbf{x})$
- Project points into some space (e.g., Euclidean)
- Center a **ball** of radius r around $\mathbf{x}$ (solid hypersphere)
- **Adjust radius** until it encloses k points in $\mathbf{X}$, i.e., $r(\mathbf{x}, k)$
- The k enclosed points $(\widehat{\mathbf{X}}, \widehat{\mathbf{y}})$ form a **neighborhood** around $\mathbf{x}$
- **Assume neighborhood describes distribution** from which $\mathbf{x}$ is drawn

Mathematical foundation (continued)

- **Density** of the distribution containing $\mathbf{x}$ is then given by

$$p(\mathbf{x}) = \frac{k}{NV}$$

  - V is volume of ball of radius $r(\mathbf{x}, k)$.
  - Note: Not a true pdf/density model (integral diverges).
  - Also doesn't really give us any intuition about algorithm.
- Assume observation $\mathbf{x}$ is reflective of its **neighborhood** $p(\mathbf{x})$.
  - Analogous to estimating slope/gradient of line at tangent in calculus.
- Known as ***k*-nearest neighbor method** of classification/regression

Common scale essential

- Common scale is essential!
- Remember: Suppose one feature reported in **m**, the other in **cm**
- **If you forget to scale, features with large values will predominate**
- GIGO!!!
- Use MinMaxScaler, StandardScaler, etc.

Scaling methods

- **Min-max scaler**
  - Scales training data $\mathbf{X}$ to [0,1]

$$\frac{\mathbf{X} - \min \mathbf{X}}{\max \mathbf{X} - \min \mathbf{X}}$$

- **Standard scaler**
  - Models as $\mathbf{X} \sim \mathrm{N}(0,1)$

$$\frac{\mathbf{X} - \bar{\mathbf{x}}}{\mathrm{std}(\mathbf{x})}$$

- **Fit parameters** (min, max, mean, std) to **training data** $\mathbf{X}$
- **THEN scale test/validation sets** using the learned parameters (why?)

Algorithm

- **Fit (train):** Store training data. Done.
- **Predict (hard part):** Iterate over n stored training examples.
- **Compute distance** from new point x to every $x^{(i)}$ in training set.
- **Keep track of** k best (smallest) **distances and indexes** during loop.
- Note: Can drop square root in implementation: **relative distance.**
- After loop: Uses k-best indexes to find corresponding true y values.
- **Which class (y value) is most common?** That's your prediction $\hat{y}$.
- **kNN-Regression:** then $\hat{y}$ is just the average: add y values, divide by k.

Tradeoffs

- Can classify data that are not linearly separable.
- Training is easy (just store the example data!).
- Easy to understand and implement.
- **Easy to explain to a human being.**
- Predictions are slow: Iterate through n training examples …
- For each of the m unclassified item in your test set: $O(nmd)$

Improving on nearest neighbor search

- Clever data structures (complete)
  - Big idea: use a tree (like binary search tree in CSC 270)
  - **kD-tree** low-dimensional data
  - **ball tree** high-dimensional data
  - scikit-learn picks one of these by default or you can specify
- **Approximate** KNN
  - **FAISS** (Facebook)
  - **ScaNN** (Google)
  - **Annoy** (Spotify) Important area of research (industry interest)
- **Vector search engines** (e.g., Weaviate uses HNSWG)

Hyperparameters

- **Parameters:** Learned by the ML algorithm
- **Hyperparameters:** Determined by the user: passed to algorithm
- kNN is a **non-parametric** model …
- But it does have some important hyperparameters.
- Can you identify any hyperparameters for kNN?
- Choice of k: number of nearest neighbors (int, usually odd)
- Also choice of distance metric: L2, L1, etc.
- **Grid search:** Do a loop over combinations, validation set, find best

Coding Demo